JVM内存结构

JVM内存结构的划分

根据Java虚拟机规范，JVM内存结构分为5个板块，又可以根据是否为线程私有分为两个部分

程序计数器

程序计数器是当前线程执行字节码的行号指示器，指向当前线程执行字节码指令地址。为了使各个线程都能在执行后回到正确的位置，因此每个线程都拥有一个程序计数器，称之为线程私有。

虚拟机栈

和程序计数器一样，Java虚拟机栈也是线程私有的，随着线程创建而生，随着线程销毁而死，不会出现线程安全问题。

虚拟机栈描述的是Java方法执行的线程内存模型。当一个线程创建时，虚拟机会为他创建一个私有的栈，每当线程执行一个方法时，就会同步创建一个栈帧，栈帧记录了每个方法的局部变量表、操作数栈、动态连接、方法出口等信息。方法执行时，栈帧入栈；方法结束后，栈帧出栈。

本地方法栈

与虚拟机栈类似，区别在于它为虚拟机内部的Native方法服务。

堆

堆是JVM内存结构中最大的一块，用于**存放对象实例**，由所有线程共享使用。堆也是JVM垃圾回收的主要区域。堆的内存在逻辑上是连续的，实际上可以是不连续的。

根据分代理论，我们将堆分为两个部分：

1. 新生代：由一个伊甸园区和两个幸存者区构成，新创建的对象会被放在伊甸园区；在GC，后会把伊甸园区和一个幸存者区中存活的对象迁移到另一个幸存者区中。
2. 老年代区：新生代的对象在经过一定次数的GC后如果仍然存活，会晋升到老年代区。

方法区

方法区也是所有线程共享使用的，它用于存储被虚拟机加载的类的信息。具体包括：类元信息、运行时常量池、静态变量以及即使编译器编译后的代码缓存等内容。

方法区经历了几个大版本的变迁：在JDK7之前，方法区被定义在堆内存当中一个称为永久代的特殊区域，容易受到堆内存的大小限制。在JDK7之后，将字符串常量池和静态变量移出了永久代，放置在了普通的堆内存中。在JDK8之后，完全放弃了永久代的概念，将方法区的剩余成员移出了永久代，放入由直接内存存储的元空间中。

字符串常量池

它是专门为 `String` 类维护的一个哈希表，为了性能优化，所有线程共享一个字符串面量相同的实例。

在JDK7之前，存放在永久代中；JDK7之后，移除永久代，直接放在堆内存当中。

运行时常量池

它是方法区的一部分，当 `.class` 文件被加载到JVM时，类文件的中的常量池表就会被加载到这里。

在JDK8之前，这个区域位于堆空间的永久代内；JDK8之后，这个区域被迁移到了由直接内存存储的元空间中。

直接内存

这一部分并不直接属于JVM内存，但是可以通过NIO进行申请用于存放数据。这一部分不会受到堆内存大小的限制，当然也不会有JVM进行垃圾回收，有可能会导致内存溢出。

对象的创建

当虚拟机遇到一条 `new` 指令时，回去先去检查这个指令是否能在**运行时常量池**中指向一个类的符号引用，来判断这个类是否被加载、初始化过；如果没有，就执行类的加载机制。

当类加载完成后，JVM就会在堆内存中申请一块内存来存放对象，申请的方式有两种：内存规整使用指针碰撞；内存不规整使用空闲列表。由于在堆上创建对象实例是非常频繁的，这也会导致线程不安全，JVM采用两种方式保证创建成功：CAS+失败重试（内存规整）或者本线程分配缓冲（内存不规整）。

内存分配完成后，就会把不包含对象头的内存初始化为零（int = 0，boolean = false，指针为null），这是保证类加载后不赋值就可以使用。

这之后就会设置对象头，设置的信息包括：指向元空间的类元数据信息的地址、GC年龄信息、对象的哈希吗（ 在执行hashCode()才进行计算 ）、锁状态等等。

执行初始化方法，执行 `init` 方法，包括成员变量的赋值、实例代码块执行、构造函数执行。